Лабораторная работа №5

Дисциплина: Обработка изображений и мультимедиа.
Тема работы: Оценка качества сжатия с использованием программы ImageMagick.
Количество часов: 2 часа.

План работы:

Для выполнения лабораторной работы №5 следует посмотреть следующие видеоролики, доступные по адресу:
https://www.youtube.com/playlist?list=PLh5xmehZOTttdw5gLfqO_swG5yNLqm9TN

1. Закодировать исходный RAW-файл "**sunset.cr2**" (файл с расширением CR2 – это формат камеры Canon) в следующие форматы:

- ✓ PNG;
- ✓ WebP без потерь;
- ✓ JPEG без потерь;
- ✓ WebP с качеством 30% и 60%;
- ✓ JPEG того же размера, что полученный WebP 60% (±10%), а также с качеством 30% и 60%.

Во всех случаях кодируется исходный RAW файл.
Все файлы будут **с глубиной 8 бит/канал**.

К сожалению, мы не сможем исследовать возможности BPG, его поддержка еще не включена в программу ImageMagick (http://imagemagick.org).

Вам будут нужны **точные размеры файлов**. Операционная система Windows имеет свойство показывать размер файла, округлив значение. Поэтому необходимо посмотреть свойства файла (там среди прочего указано **число в байтах**), либо воспользоваться командой DIR, раз уж все равно работа происходит в командной строке.

Вы удивитесь разнице в качестве, именно за этим нам пригодятся запасные JPEG и WebP (30% и 60%). Из них мы посчитаем разницу в размере.

Не спешите на глазок подбирать уровень качества в JPEG по аналогии с указанием "**-quality 60**" для WebP. Для этого есть более подходящий ключ (http://www.imagemagick.org/Usage/formats/#jpg_write), если вы уже знаете размер, к которому стремитесь. При таком сжатии точного совпадения размера не будет, поэтому допускается погрешность 10%. Просто вводите в настройках кодирования то число килобайт, которое занимает соответствующий файл WebP.

По той же ссылке найдете, как кодировать Lossless JPEG. Будет полезно для осознания возможностей кодирования, если вы ознакомитесь с другими ключами.

Совет: создайте пустую папку, положите туда RAW-файл. Создайте в той же папке текстовый документ, дайте ему расширение ".cmd" и записывайте туда команды, каждую с новой строки. В последней строке напишите "Pause". Сохраните файл, запустите его двойным кликом – это командный файл Windows. Он выполнит последовательно все написанные команды и оставит открытой консоль, пока вы не нажмете любую кнопку (чтобы видеть ошибки, если они будут). Это сэкономит вам массу времени. Для отладки, чтобы не ждать выполнения уже работающих команд, вы можете закомментировать на время уже работающие строки командой REM – они будут игнорироваться.

Важно: если в команде содержится знак "%", то при помещении такой команды в файл пакетной обработки нужно добавить туда еще один знак "%". Это служебный символ, и он требует «экранирования», иначе выполняет другую роль. Аналогично замените одинарные кавычки-

апострофы (‘) на двойные (“) – с одинарными в некоторых случаях бывают сложности при пакетной обработке.

2. Сравнить (получить файл-разницу) попарно:
- ✓ PNG – WebP lossless;
- ✓ PNG – JPEG lossless;
- ✓ RAW – WebP lossy (WebP 60%);
- ✓ RAW – JPEG lossy (JPEG, созданный по размеру WebP 60%);
- ✓ WebP lossy – JPEG lossy (WebP 60% и соответствующий ему по размеру JPEG).

Для этого вам понадобится всего одна команда, в которую вы будете подставлять нужные имена файлов. Из этой страницы (http://www.imagemagick.org/Usage/compare/) можно подчерпнуть такую строку:

```
convert file1 file2 -compose difference -composite file.png
```

Заранее продумайте логику именования файлов, чтобы не запутаться.
Обратите внимание: ключ **`-colorspace Gray`**, который Вы увидите в соответствующем видеоролике, мы использовать не будем, чтобы не терять точность.

Заметьте, не все изначально полученные сжатые файлы будут участвовать в сравнении вышеописанным методом. Например, JPEG-30%, JPEG-60%, WebP-30% ***не будут*** *участвовать в сравнении.*

3. Дальше потребуется изменить размер файлов-разниц. Здесь важно указать размер, а интерполяцию, то есть, способ вычисления значений пикселей, в нашем случае указывать не обязательно.

4. Вычислить ошибку для каждого файла и обобщить результаты. Вот полезная информация о том, как узнать значение пикселя из командной строки (http://www.imagemagick.org/discourse-server/viewtopic.php?t=19297). Впрочем, об этом – ближе к делу, а пока нужно сделать рабочие файлы из исходной RAW-фотографии.

Среди файлов, которые вы сгенерировали по инструкции из прошлого шага, должны быть следующие:
- ✓ JPEG 30%;
- ✓ JPEG 60%;
- ✓ WebP 30%;
- ✓ WebP 60%.

Посчитайте средние значения размеров для WebP и JPEG в этом примере и напишите их отношение (меньшее поделите на большее) – получится доля (число от 0 до 1). Переведите в проценты (умножьте на 100).

PS: вы так же сделали JPEG, соответствующий по размеру WebP 60%. С ним мы будем работать дальше.

Из исходного RAW-файла вы сделали три lossless-изображения:
- ✓ PNG;
- ✓ JPEG-lossless;

- ✓ WebP-lossless.

Введите точный размер в байтах* самого маленького из этих трех файлов.

* В связи с особенностями работы ImageMagick на разных компьютерах, ответы принимаются в довольно широком диапазоне. Особое внимание обратите на кодирование JPEG Lossless – там важно указывать и режим, и качество.

Если вы еще не сделали файлы-разницы, то теперь самое время их сделать. Напомню, нам потребуется сравнить следующие файлы:

- ✓ PNG – WebP lossless;
- ✓ PNG – JPEG lossless;
- ✓ RAW – WebP lossy (WebP 60%);
- ✓ RAW – JPEG lossy (JPEG, созданный по размеру WebP 60%);
- ✓ WebP lossy – JPEG lossy (WebP 60% и соответствующий ему по размеру JPEG).

Обратите внимание, что JPEG-30%, JPEG-60%, WebP-30% не будут участвовать в сравнении.

Итого, должно получиться пять файлов. Напомню, мы кодируем цветное дифференциальное изображение (ключ **`-colorspace Gray`** не нужен).

Обратите внимание на размеры этих файлов. Учитывая, что это – файлы ошибок и исследуется реальное, а не специально заготовленное синтетическое изображение, эти размеры сами по себе характеризуют разницу. Но здесь может играть роль алгоритм сжатия или еще какие-то факторы, так что мы двинемся дальше, а в этом задании **укажите отношение размера файлов ошибки для первых двух (lossless) файлов** (большее число делите на меньшее).

Допускается погрешность ±5% от заданного «эталонного» результата.

Результат должен навести вас на мысли о странном поведении одного из этих форматов. С этими мыслями перейдем к самому интересному: получению значения ошибки в явном виде.

Наличие файла с отмеченными ошибками кодирования само по себе не дает возможности узнать, какой же общий уровень разницы был получен при кодировании сравниваемых файлов.

Для этого нужно провести две операции:

1. Уменьшить файл-разницу до размеров 1×1 пиксель
2. Узнать значение этого пикселя.

Для получения результатов в этом и следующем шаге вам пригодится или блокнот с калькулятором или, лучше, электронная таблица (Excel, Google Spreadsheets и т.д.).

Итак, **уменьшите все полученные файлы-разницы до размера 1×1 пиксель.** Интерполяцию можно не указывать, она по умолчанию правильная.

Дальше есть одна тонкость: можно было бы открыть полученное изображение в фоторедакторе, посмотреть значение пикселя и его записать. Только, к сожалению, фоторедакторы в наше непростое время настолько круты, что значения пикселей домысливают по-своему. Например, Photoshop, Paint и онлайн-редактор Pixlr выдали разные наборы значений для одного файла. В обычной жизни это нам бы вряд ли помешало (цветовые профили там мешаются, или они налоги с яркости собирают), но сейчас хотелось бы однозначности. И поэтому мы снова обратимся к ImageMagick (http://www.imagemagick.org/Usage/quantize/#extract).

Вот строка, которая вам поможет получить значения цвета.

```
convert file.png -format '%[pixel:p{0,0}]' info:
```

На выходе будет или gray(0), или значения sRGB в процентах, или значения sRGB в уровнях по шкале от 0 до 255. Переведите все в проценты и запишите себе в табличку прямо со всеми указанными знаками после запятой. Для “gray(0)” – везде нули.

Посчитайте **среднее арифметическое,** округлите его до **тысячных**.

Вот теперь можно сравнить результаты: у нас есть понятные числа для этого.

Например, вы помните, что файл-разница JPEG Lossless относительно PNG был какой-то слишком большой, если сравнивать с аналогичным файлом-разницей для WebP Lossless? А теперь посмотрите, в чем конкретно проявляется ошибка. При желании можно взять этот файл, выделить из него сбойный канал и в нем увидеть, где конкретно возникает ошибка кодирования.

В ответ на этот вопрос введите **отношение процента ошибки между 3 и 4 файлами** (это сравнение с RAW WebP 60% и JPEG соответствующего размера). Большее значение делите на меньшее, **округляйте до сотых.**

Во столько раз WebP меньше мусорит при сжатии, чем JPEG того же размера.

На всякий случай допускается погрешность.